LESSON 01

First Delta Lake table

Managed Delta customers and orders in one demo schema. Read with SQL and PySpark, update one fictional row, then time-travel to the version captured before that update.

Safe writes

- Schema byte_delta_demo_20261106 in the current catalog only. The setup cell stops if that name or the table names change.
- CREATE SCHEMA IF NOT EXISTS and CREATE TABLE IF NOT EXISTS … USING DELTA. No LOCATION and no /mnt path.
- A re-run DELETEs rows in the demo customers or orders table, then inserts the sample rows again. It does not DROP or CREATE OR REPLACE.
- UPDATE touches only customer_id = 2 with a SAMPLE data note (Alan, Manchester to Cambridge).
- pre_update_version is a Python int: max(version) from DESCRIBE HISTORY before the update. Do not hard-code it, and do not call spark.conf.set.

```
DEMO_SCHEMA = "byte_delta_demo_20261106"
CATALOG = spark.sql(
    "SELECT current_catalog() AS catalog"
).collect()[0]["catalog"]
schema_fqn = f"`{CATALOG}`.`{DEMO_SCHEMA}`"
customers = f"{schema_fqn}.`customers`"
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema_fqn}")
spark.sql(f"""
CREATE TABLE IF NOT EXISTS {customers} (
  customer_id INT, name STRING, city STRING,
  signup_date DATE, data_note STRING
) USING DELTA
""")
```

Sample customers, then one update

Three fictional rows. data_note is SAMPLE — fictional. After the load, Alan is still in Manchester.

id	name	city before	city after update
1	Ada Lovelace	London	London
2	Alan Turing	Manchester	Cambridge
3	Grace Hopper	New York	New York

```
spark.sql(f"DELETE FROM {customers}")  # demo table only
# then INSERT the three SAMPLE rows, including DATE '1815-12-10'
```

Reads

```
display(spark.sql(f"""
SELECT customer_id, name, city, signup_date, data_note
FROM {customers}
ORDER BY customer_id
"""))
display(spark.table(customers).orderBy("customer_id"))
display(spark.table(customers).filter("city = 'London'"))  # Ada
```

Capture the version, then update

DELETE and INSERT add commits, so the version number changes every load. Read max(version) first, store it in pre_update_version, then update.

```
hist = spark.sql(f"DESCRIBE HISTORY {customers}")
pre_update_version = int(
    hist.selectExpr("max(version) AS v").collect()[0]["v"]
)
spark.sql(f"""
UPDATE {customers}
SET city = 'Cambridge'
WHERE customer_id = 2
  AND data_note LIKE 'SAMPLE%'
""")
```

Time travel back to that version

```
spark.sql(f"""
SELECT customer_id, name, city, data_note
FROM {customers} VERSION AS OF {pre_update_version}
ORDER BY customer_id
""")
df_prior = (
    spark.read.format("delta")
    .option("versionAsOf", str(pre_update_version))
    .table(customers)
)
```

What to check

- SQL and PySpark both show three rows. The London filter shows Ada only.
- After UPDATE, Alan's city is Cambridge. Ada and Grace stay put. VERSION AS OF the captured number still shows Manchester.
- DESCRIBE HISTORY includes an UPDATE. The printed version is whatever max(version) was on this run.
- Orders practice uses the same schema. Amounts are CAST(n AS DOUBLE) because serverless ANSI SQL treats a bare 42.50 as DECIMAL. Ada appears twice. Grace has no order.

```
spark.sql(f"""
INSERT INTO {orders} VALUES
  (1001, 1, CAST(42.50 AS DOUBLE), 'SAMPLE — fictional'),
  (1002, 2, CAST(19.99 AS DOUBLE), 'SAMPLE — fictional'),
  (1003, 1, CAST(7.25 AS DOUBLE), 'SAMPLE — fictional')
""")
```